Project One
CS 300
ABCU Computer Science Advising Program
Brandon Witbrot

Pseudocode

1. Milestone One: Vector

Course Structure

```
STRUCT Course
String courseNumber
String title
Vector<String> prerequisites
END STRUCT
```

Helper Function: Determine Whether a Course Exists

```
Boolean courseExists(Vector<Course> courses, String courseNumber)
FOR each course IN courses
IF course.courseNumber equals courseNumber
RETURN true
END IF
END FOR
RETURN false
END FUNCTION
```

Load, Parse, Validate, and Store the Course File

This routine validates each line while it is read, then validates prerequisites after all course objects

have been stored so prerequisites may refer to courses appearing later in the file.

```
Vector<Course> loadCourses(String fileName)
CREATE empty Vector<Course> courses
SET formatError = false
SET lineNumber = 0
OPEN fileName for input
IF file does not open successfully
PRINT "Error: Unable to open course data file."
RETURN empty courses
END IF
WHILE there is another line in the file
READ next line into line
INCREMENT lineNumber
SPLIT line at each comma into Vector<String> tokens
TRIM leading and trailing spaces from every token
// Every line must contain at least course number and title.
IF tokens.size is less than 2
PRINT "Format error on line " + lineNumber + ": fewer than two parameters."
SET formatError = true
CONTINUE to next line
```

```
END IF
IF tokens[0] is empty OR tokens[1] is empty
PRINT "Format error on line " + lineNumber + ": course number or title is missing."
SET formatError = true
CONTINUE to next line
END IF
CREATE Course currentCourse
SET currentCourse.courseNumber = tokens[0]
SET currentCourse.title = tokens[1]
CREATE empty currentCourse.prerequisites
// Tokens after the first two are prerequisite course numbers.
FOR index = 2 TO tokens.size - 1
IF tokens[index] is not empty
ADD tokens[index] TO currentCourse.prerequisites
END IF
END FOR
ADD currentCourse TO courses
END WHILE
CLOSE file
IF formatError equals true
CLEAR courses
RETURN courses
END IF
// Validate every prerequisite after all course numbers are available.
FOR each course IN courses
FOR each prerequisiteNumber IN course.prerequisites
IF courseExists(courses, prerequisiteNumber) equals false
PRINT "Format error: prerequisite " + prerequisiteNumber +
" for " + course.courseNumber + " does not exist in the file."
SET formatError = true
END IF
END FOR
END FOR
IF formatError equals true
CLEAR courses
RETURN courses
END IF
PRINT "Course data loaded successfully."
RETURN courses
END FUNCTION
```

Search for a Course and Print Its Information

The search checks each course in the vector. When a match is found, it prints the selected course and then prints the course number and title for each prerequisite.

```
Void searchCourse(Vector<Course> courses, String courseNumber)
FOR each course IN courses
IF course.courseNumber equals courseNumber
```

```
PRINT course.courseNumber + ", " + course.title
IF course.prerequisites is empty
PRINT "Prerequisites: None"
ELSE
PRINT "Prerequisites:"
FOR each prerequisiteNumber IN course.prerequisites
FOR each prerequisiteCourse IN courses
IF prerequisiteCourse.courseNumber equals prerequisiteNumber
PRINT prerequisiteCourse.courseNumber + ", " + prerequisiteCourse.title
BREAK inner loop
END IF
END FOR
END FOR
END IF
RETURN
END IF
END FOR
PRINT "Course not found."
END FUNCTION
```

Program Flow Using the Vector

```
MAIN
SET fileName = "course_data.txt"
SET courses = loadCourses(fileName)
IF courses is empty
PRINT "No course data is available. Correct the file and try again."
STOP program
END IF
PROMPT user to enter a course number
READ courseNumber
CALL searchCourse(courses, courseNumber)
END MAIN
```

Print Course List

```
Void printCourseList(Vector<Course> courses)
  CREATE Vector<Course> sortedCourses = copy of courses
  SORT sortedCourses by courseNumber in ascending alphanumeric order
  FOR each course IN sortedCourses
    PRINT course.courseNumber + ", " + course.title
  END FOR
END FUNCTION
```

2. Milestone Two: Hash Table

Course Structure

```
STRUCT Course
```

```
String courseNumber
String title
Vector<String> prerequisites
END STRUCT
```

Helper Function: Determine Whether a Course Exists

```
Boolean courseExists(HashTable<Course> courses, String courseNumber)
SET foundCourse = courses.search(courseNumber)
IF foundCourse exists
RETURN true
END IF
RETURN false
END FUNCTION
```

Load, Parse, Validate, and Store the Course File

This routine validates each line while it is read, then validates prerequisites after all course objects have been stored so prerequisites may refer to courses appearing later in the file.

```
HashTable<Course> loadCourses(String fileName)
CREATE empty HashTable<Course> courses
SET formatError = false
SET lineNumber = 0
OPEN fileName for input
IF file does not open successfully
PRINT "Error: Unable to open course data file."
RETURN empty courses
END IF
WHILE there is another line in the file
READ next line into line
INCREMENT lineNumber
SPLIT line at each comma into Vector<String> tokens
TRIM leading and trailing spaces from every token
// Every line must contain at least course number and title.
IF tokens.size is less than 2
PRINT "Format error on line " + lineNumber + ": fewer than two parameters."
SET formatError = true
CONTINUE to next line
END IF
IF tokens[0] is empty OR tokens[1] is empty
PRINT "Format error on line " + lineNumber + ": course number or title is missing."
SET formatError = true
CONTINUE to next line
END IF
CREATE Course currentCourse
SET currentCourse.courseNumber = tokens[0]
SET currentCourse.title = tokens[1]
CREATE empty currentCourse.prerequisites
```

```
// Tokens after the first two are prerequisite course numbers.
FOR index = 2 TO tokens.size - 1
IF tokens[index] is not empty
ADD tokens[index] TO currentCourse.prerequisites
END IF
END FOR
INSERT currentCourse INTO courses USING currentCourse.courseNumber AS key
END WHILE
CLOSE file
IF formatError equals true
CLEAR courses
RETURN courses
END IF
// Validate every prerequisite after all course numbers are available.
FOR each course IN courses
FOR each prerequisiteNumber IN course.prerequisites
IF courseExists(courses, prerequisiteNumber) equals false
PRINT "Format error: prerequisite " + prerequisiteNumber +
" for " + course.courseNumber + " does not exist in the file."
SET formatError = true
END IF
END FOR
END FOR
IF formatError equals true
CLEAR courses
RETURN courses
END IF
PRINT "Course data loaded successfully."
RETURN courses
END FUNCTION
```

Search for a Course and Print Its Information

The search uses the course number as the hash table key. When a match is found, it prints the selected course and then prints the course number and title for each prerequisite.

```
Void searchCourse(HashTable<Course> courses, String courseNumber)
SET course = courses.search(courseNumber)
IF course does not exist
PRINT "Course not found."
RETURN
END IF
PRINT course.courseNumber + ", " + course.title
IF course.prerequisites is empty
PRINT "Prerequisites: None"
ELSE
PRINT "Prerequisites:"
FOR each prerequisiteNumber IN course.prerequisites
SET prerequisiteCourse = courses.search(prerequisiteNumber)
```

```
IF prerequisiteCourse exists
PRINT prerequisiteCourse.courseNumber + ", " + prerequisiteCourse.title
END IF
END FOR
END IF
END FUNCTION
```

Program Flow Using the Hash Table

```
MAIN
SET fileName = "course_data.txt"
SET courses = loadCourses(fileName)
IF courses is empty
PRINT "No course data is available. Correct the file and try again."
STOP program
END IF
PROMPT user to enter a course number
READ courseNumber
CALL searchCourse(courses, courseNumber)
END MAIN
```

Print Course List

```
Void printCourseList(HashTable<Course> courses)
  CREATE empty Vector<Course> sortedCourses
  FOR each course IN courses
    ADD course TO sortedCourses
  END FOR
  SORT sortedCourses by courseNumber in ascending alphanumeric order
  FOR each course IN sortedCourses
    PRINT course.courseNumber + ", " + course.title
  END FOR
END FUNCTION
```

3. Milestone Three: Binary Search Tree

Course Structure

```
STRUCT Course
  String courseNumber
  String title
  Vector<String> prerequisites
END STRUCT
```

Binary Search Tree Structure

```
STRUCT TreeNode
  Course course
  TreeNode left
  TreeNode right
```

```
  END STRUCT
  STRUCT BinarySearchTree
    TreeNode root
  END STRUCT
```

Insert a Course into the Binary Search Tree

```
  TreeNode insertCourse(TreeNode node, Course course)
    IF node is null
      CREATE new TreeNode newNode
      SET newNode.course = course
      SET newNode.left = null
      SET newNode.right = null
      RETURN newNode
    END IF
   IF course.courseNumber is less than node.course.courseNumber
     SET node.left = insertCourse(node.left, course)
   ELSE IF course.courseNumber is greater than node.course.courseNumber
     SET node.right = insertCourse(node.right, course)
   ELSE
     // Course number already exists. Do not insert a duplicate node.
     RETURN node
   END IF
    RETURN node
  END FUNCTION
```

Search the Binary Search Tree for a Course

```
  Course searchCourse(BinarySearchTree courses, String courseNumber)
    SET current = courses.root
   WHILE current is not null
     IF courseNumber equals current.course.courseNumber
       RETURN current.course
     ELSE IF courseNumber is less than current.course.courseNumber
       SET current = current.left
      ELSE
       SET current = current.right
     END IF
   END WHILE
    RETURN null
  END FUNCTION
  Boolean courseExists(BinarySearchTree courses, String courseNumber)
    SET foundCourse = searchCourse(courses, courseNumber)
    IF foundCourse exists
      RETURN true
    END IF
    RETURN false
  END FUNCTION
```

Validate Prerequisites in the Tree

This recursive traversal checks every stored course after the entire file has been loaded, allowing prerequisites to refer to courses that appear later in the input file.

```
Boolean validatePrerequisites(TreeNode node, BinarySearchTree courses)
  IF node is null
    RETURN true
  END IF
  SET valid = true
  IF validatePrerequisites(node.left, courses) equals false
    SET valid = false
  END IF
  FOR each prerequisiteNumber IN node.course.prerequisites
    IF courseExists(courses, prerequisiteNumber) equals false
      PRINT "Format error: prerequisite " + prerequisiteNumber +
        " for " + node.course.courseNumber + " does not exist in the file."
      SET valid = false
    END IF
  END FOR
  IF validatePrerequisites(node.right, courses) equals false
    SET valid = false
  END IF
  RETURN valid
END FUNCTION
```

Load, Parse, Validate, and Store the Course File

```
BinarySearchTree loadCourses(String fileName)
  CREATE empty BinarySearchTree courses
  SET courses.root = null
SET formatError = false
  SET lineNumber = 0
  OPEN fileName for input
  IF file does not open successfully
    PRINT "Error: Unable to open course data file."
    RETURN empty courses
  END IF
  WHILE there is another line in the file
    READ next line into line
    INCREMENT lineNumber
     SPLIT line at each comma into Vector<String> tokens
     TRIM leading and trailing spaces from every token
     // Every line must contain at least a course number and title.
     IF tokens.size is less than 2
       PRINT "Format error on line " + lineNumber + ": fewer than two
       parameters."
       SET formatError = true
       CONTINUE to next line
```

```
      END IF
      IF tokens[0] is empty OR tokens[1] is empty
        PRINT "Format error on line " + lineNumber +
            ": course number or title is missing."
        SET formatError = true
        CONTINUE to next line
      END IF
      CREATE Course currentCourse
      SET currentCourse.courseNumber = tokens[0]
      SET currentCourse.title = tokens[1]
      CREATE empty currentCourse.prerequisites
      // Tokens after the first two are prerequisite course numbers.
      FOR index = 2 TO tokens.size - 1
        IF tokens[index] is not empty
          ADD tokens[index] TO currentCourse.prerequisites
        END IF
      END FOR
      SET courses.root = insertCourse(courses.root, currentCourse)
    END WHILE
    CLOSE file
    IF formatError equals true
      CLEAR courses
      RETURN courses
    END IF
    // Validate prerequisites only after all course numbers are in the tree.
    IF validatePrerequisites(courses.root, courses) equals false
      CLEAR courses
      RETURN courses
    END IF
    PRINT "Course data loaded successfully."
    RETURN courses
  END FUNCTION
```

Print All Courses in Alphanumeric Order

An in-order traversal prints the binary search tree in ascending course-number order.

```
  Void printCourseList(TreeNode node)
    IF node is null
      RETURN
    END IF
    CALL printCourseList(node.left)
    PRINT node.course.courseNumber + ", " + node.course.title
    CALL printCourseList(node.right)
  END FUNCTION
```

Print Course Information and Prerequisites

```
  Void printCourseInformation(BinarySearchTree courses, String courseNumber)
    SET course = searchCourse(courses, courseNumber)
```

```
  IF course does not exist
    PRINT "Course not found."
    RETURN
  END IF
  PRINT course.courseNumber + ", " + course.title
  IF course.prerequisites is empty
    PRINT "Prerequisites: None"
  ELSE
    PRINT "Prerequisites:"
    FOR each prerequisiteNumber IN course.prerequisites
      SET prerequisiteCourse = searchCourse(courses, prerequisiteNumber)
       IF prerequisiteCourse exists
         PRINT prerequisiteCourse.courseNumber + ", " +
         prerequisiteCourse.title
       END IF
     END FOR
   END IF
END FUNCTION
```

Program Flow Using the Binary Search Tree

```
MAIN
  SET fileName = "course_data.txt"
  SET courses = loadCourses(fileName)
 IF courses.root is null
    PRINT "No course data is available. Correct the file and try again."
    STOP program
  END IF
  PRINT "Course List:"
  CALL printCourseList(courses.root)
   PROMPT user to enter a course number
   READ courseNumber
   CALL printCourseInformation(courses, courseNumber)
END MAIN
```

Print Course List

The previously submitted recursive printCourseList(TreeNode node) already performs an in-order traversal, so no replacement is necessary.

4. Menu-Driven Program

This menu replaces the immediate-load MAIN routines shown in the three milestones. Run it with one selected data structure; printCourseInformation for the tree and searchCourse for the vector and hash table.

```
MAIN
  CREATE empty courses using the selected data structure
```

```
  SET loaded = false
  REPEAT
    PRINT "1. Load Data Structure"
    PRINT "2. Print Course List"
    PRINT "3. Print Course"
    PRINT "9. Exit"
    READ choice
    IF choice equals 1
      PROMPT for fileName
      SET courses = loadCourses(fileName)
      SET loaded = courses is not empty
    ELSE IF choice equals 2
      IF loaded equals false
        PRINT "Load course data first."
      ELSE IF using binary search tree
        CALL printCourseList(courses.root)
      ELSE
        CALL printCourseList(courses)
      END IF
    ELSE IF choice equals 3
      IF loaded equals false
        PRINT "Load course data first."
      ELSE
        PROMPT for courseNumber
        READ courseNumber
        IF using binary search tree
          CALL printCourseInformation(courses, courseNumber)
        ELSE
          CALL searchCourse(courses, courseNumber)
        END IF
      END IF
    ELSE IF choice is not 9
      PRINT "Invalid option."
    END IF
  UNTIL choice equals 9
  PRINT "Goodbye."
END MAIN
```

5. Runtime Analysis

Let n be the number of courses and p the maximum prerequisites per course. Following the course data, assume p and the length of each CSV record are bounded constants; split, trim, and creation therefore take O(1) per record. Each simple statement costs 1; function calls use their operation cost. The figures below analyze loading, insertion, and post-load prerequisite validation—not menu execution, searching for display, or printing the sorted list.

Vector: Worst-Case Loading

Pseudocode operation	Cost per execution	Times executed	Total cost
Initialize and open/close file	1	constant	O(1)
Read, split, trim, check fields	1 per bounded record	n	O(n)
Create course and prerequisite vector	1 per bounded record	n	O(n)
Append course to vector	amortized O(1)	n	O(n) amortized
Loop over courses and prerequisites	1 per check	$\leq np$	O(np)
courseExists linear scan	O(n)	$\leq np$	$O(n^2p)$
Report errors, clear, return	1 / O(n) clear	at most once	O(n)

With bounded p, worst-case total $T(n) = O(n^2)$; auxiliary storage for course objects is O(n).

Hash Table: Worst-Case Loading

Pseudocode operation	Cost per execution	Times executed	Total cost
Initialize and open/close file	1	constant	O(1)
Read, split, trim, check fields	1 per bounded record	n	O(n)
Create course and prerequisite vector	1 per bounded record	n	O(n)
Insert by courseNumber	O(n) worst collision case	n	$O(n^2)$
Loop over courses and prerequisites	1 per check	$\leq np$	O(np)
courseExists / hash search	O(n) worst collision case	$\leq np$	$O(n^2p)$
Report errors, clear, return	1 / O(n) clear	at most once	O(n)

With bounded p, worst-case total $T(n) = O(n^2)$; expected total is O(n) with effective hashing, controlled load factor, and expected O(1) insert/search. Storage is O(n).

Binary Search Tree: Worst-Case Loading

Pseudocode operation	Cost per execution	Times executed	Total cost
Initialize root and open/close file	1	constant	O(1)
Read, split, trim, check fields	1 per bounded record	n	O(n)
Create course and	1 per bounded record	n	O(n)

prerequisite vector			
Recursive insertCourse	O(n) when skewed	n	$O(n^2)$
Traverse stored tree for validation	1 per node	n	O(n)
courseExists / searchCourse	O(n) when skewed	$\leq np$	$O(n^2p)$
Report errors, clear, return	1 / O(n) clear	at most once	O(n)

With bounded p, worst-case total $T(n) = O(n^2)$; an approximately balanced tree gives O(n log n) loading and validation. Stored nodes and courses require O(n), with O(n) worst-case recursion/height overhead.

6. Evaluation and Recommendation

Vector: Appending courses is straightforward, compact, and efficient, but searching for each prerequisite requires a linear scan. Loading with prerequisite validation is $O(n^2)$, and printing the full list requires an additional O(n log n) sort. Memory usage is O(n).

Hash table: Searching by course number is expected O(1), so loading and prerequisite checking are expected O(n). However, collisions can degrade searches and insertion to O(n), producing an $O(n^2)$ worst case. Entries are not stored in alphanumeric order, so option 2 requires collecting keys and sorting them in O(n log n) time. Memory is O(n), with extra bucket capacity.

Binary search tree: The existing in-order printCourseList produces an ordered list in O(n) without a separate sort. A reasonably balanced tree supports O(log n) insert/search and O(n log n) loading, but the submitted unbalanced implementation can become skewed, with O(n) insert/search and $O(n^2)$ total worst-case loading. Nodes require pointer overhead; storage is O(n).

Recommendation: Use the binary search tree from Milestone Three for Project Two. The advisor needs an alphanumerically ordered course list as well as an individual-course lookup, and the already-written in-order traversal directly meets the ordering requirement. This choice is most attractive when insertion order keeps the tree reasonably balanced; in the worst case, the hash table performs no worse asymptotically and has better expected lookups. A self-balancing tree would eliminate the skew-related performance risk, but that would be an enhancement beyond the submitted milestone design.